A3 + The Slop Feedback Loop

Formal Methods Deep Dive (Private Notes)

Halley Young

March 2026

1. Deliverable first

Slide 1 2 min **Script:**

- Open with what ships, not with history: `pip install a3-python` gives scan + static elimination + optional triage.
- The architecture is explicitly static-first, agentic-second.
- Stress asymmetry: formal methods take the bulk volume; LLM handles uncertainty residue only.
- Set expectation for RISE audience: this talk is about what is proved, what is approximated, and where assumptions live.
- Keep phrase: "proof/disproof artifacts are upstream; triage is downstream policy support."

2. Blog arc we follow

Slide 2 2 min **Script:**

- Follow the blog storyline directly: origin challenge → Episode I/II/III → shipping architecture.
- Then expand on RISE-sensitive details: transition semantics, obligations, solver outcomes, refinement policy.
- End on operational policy: kitchensink orchestration, residual triage, CI ratchet.
- Frame from the start: this is verification-oriented systems engineering, not theorem theater.

3. One ridiculous origin story

Slide 3 2 min **Script:**

- Blog claim: this started as "can AI generate paper-scale verification ideas that survive code?"
- Early theory looked elegant but collapsed on implementation details.
- Working rule became adversarial: every idea must survive tests and regression runs.
- Useful phrase from blog: "fight AI slop with AI slop" $\rightarrow$ generate aggressively, filter brutally.
- This is why the stack values executable semantics above conceptual novelty.

4. Episode I: quantitative safety intuition

Slide 4 2 min **Script:**

- Distance semantics survived as prioritization intuition, not final correctness engine.
- Keep distinction explicit: ranking answers “what first,” verification answers “what never.”
- This sets up Episode II naturally: separation proofs for reachability exclusion.
- If challenged, concede limits of pure metrics for industrial false-positive suppression.

5. Episode II: prove unreachable

Slide 5 2 min **Script:**

- Reachability framing: show $R \cap U = \varnothing$ under encoded semantics and assumptions.
- Barrier-style certificates matter because they summarize many executions with inductive constraints.
- They are only meaningful when tied to concrete state projections from actual Python execution points.
- Failure is informative: obligation counterexamples feed refinement and portfolio rerouting.
- Good caveat: certificates are powerful but not universally synthesizeable under budget.

6. Barrier obligations (informal form)

Slide 6 2 min **Script:**

- Initialization: $\forall s \in S_0,\ B(s) \geq \epsilon$.
- Consecution: $\forall s \to s',\ B(s) \geq 0 \Rightarrow B(s') \geq 0$.
- Unsafe separation: $\forall s \in U,\ B(s) \leq -\epsilon$.
- Emphasize conjunctive acceptance: one failed obligation means no certificate.
- Solver policy is conservative: UNKNOWN does not justify pruning.
- RISE point: this is a soundness-preserving operational policy, not convenience behavior.

6b. Exact workflow: one polynomial certificate, one Python bug

Slide 6b

3 min **Script:**

- **Bug anchor:** pick one reported site, e.g., DIV_ZERO at `f` offset o for `n / d`.
- **State basis:** choose symbolic variables x from locals/stack/heap observers at o; compute path precondition $\Phi(x)$.
- **Unsafe set:** $U(x) \equiv (d(x) = 0) \wedge \Phi(x)$.
- **Dynamics:** derive entry set S_0 and transition relation $T(x, x')$ from bytecode transfer semantics.
- **Template:** choose degree-k polynomial $B(x) = \sum_{\alpha \in \mathcal{M}_k} c_\alpha x^\alpha$ with unknown coefficients c_α.
- **Obligations:** init $\forall x \in S_0 : B(x) \geq \epsilon$; consecution $\forall x, x' : T \wedge B(x) \geq 0 \Rightarrow B(x') \geq 0$; unsafe $\forall x : U(x) \Rightarrow B(x) \leq -\epsilon$.
- **Solve:** synthesize c_α via staged backends (fast checks first, stronger relaxations next).
- **Validate:** replay obligations on current model; any counterexample rejects candidate certificate.
- **Mini-example:** if Python guard is `if den > 0: return num/den`, choose $x = den$, $U(x) \equiv (x = 0)$, and template $B(x) = x - \frac{1}{2}$. Then init/consecution hold under guard-preserving transitions and unsafe separation holds since $B(0) = -\frac{1}{2} < 0$.
- **Map back:** valid certificate marks this exact site unreachable (FP) with proof artifact: template, coefficients, obligations, site metadata.
- **Fallback:** no certificate within budget $\neq$ safety proof; candidate escalates to induction/CHC/PDR/concolic/triage.
- **Q&A:** source mapping uses file/function/offset+symbol bindings; abstraction errors are caught by obligation counterexamples.

7. Episode III: executable Python semantics

Slide 7 2 min **Script:**

- Blog Episode III is the key turn: stop hand-waving and model CPython behavior explicitly.
- Include exception paths, call effects, and unknown behavior assumptions as first-class semantics.
- Representation split is intentional: concrete control, symbolic data/feasibility.
- Avoid overclaim: this is executable approximation with explicit assumptions, not full interpreter equivalence.

8. Blog technical example: assertion escape

Slide 8 2 min **Script:**

- Claim: "failing assertion escapes uncaught" is a reachability property, not a pattern match.
- Must account for assert mode, local handlers, caller handlers, and `finally` propagation.
- Naive rule-based detection over-reports; theorem-only abstractions under-specify behavior.
- The loop resolved this by combining executable semantics with conservative obligations.
- This is the blog's concrete argument for theory+code co-evolution.

9. Transition system used by the analyzer

Slide 9 2 min **Script:**

- State tuple: $\sigma = (pc, \rho, h, \tau, \kappa, e, \Phi)$.
- pc and frame-stack control are concrete; branching uncertainty lives in Φ.
- ρ, h carry symbolic values and heap observers for solver-visible reasoning.
- Exception mode and handler context are explicit; this is critical for uncaught-escape correctness.
- Unsafe predicates are bug-family-indexed: $U_t(\sigma)$.

10. Real engine from the blog: theorize → code → test → fix code → fix theory

Slide 10 2 min **Script:**

- This loop is not process fluff; it is the quality control mechanism.
- "Fixing theory" means tightening assumptions and obligations when tests expose semantic mismatch.
- Emphasize falsification pressure from synthetic suites and real-repo scans.
- Good soundbite: elegant ideas are cheap; executable invariants are expensive.

11. What shipped: static-first, agentic-second

Slide 11 2 min **Script:**

- Static stage handles the volume: discovery, feasibility, proofs, elimination, evidence packing.
- Agentic stage handles survivors only, with repository context exploration.
- Explicitly separate theorem-status artifacts from confidence/rationale artifacts.
- This asymmetry is both a trust design and a scaling design.

12. Kitchensink portfolio: blog principle in formal-method terms

Slide 12 2 min **Script:**

- Blog wording: "steal the best ideas, orchestrate them, don't worship one paper."
- Translation: route bug shapes through complementary proving families.
- Typical route: guard+SMT $\rightarrow$ barrier attempts $\rightarrow$ induction/CHC/PDR $\rightarrow$ concolic confirmation.
- Keep disagreement visible; it is diagnostic signal for abstraction quality.
- Transition to next two slides: explain **why** this composition is coherent.

12b. Barrier obligations as the unifying interface

Slide 12b 3 min **Script:**

- The barrier certificate imposes three provable obligations: init, consecution, unsafe separation.
- These obligations are not just a proof artifact — they are an **API**.
- Each paper from the 20-paper portfolio is a specialised subsolver for one or more of those obligations.
- The framework is not "use everything and hope"; it is structured decomposition of a single verification problem.
- RISE point: the semantic contract (B with three conditions) is fixed; methods compete to discharge each condition.
- Method failure is informative: if SOS cannot certify separation, that tells you the unsafe region boundary is outside polynomial reach for the current variable basis.
- That failure is then a routing signal, not a dead end.

12c. Separation obligation: foundations (papers 1–5)

Slide 12c 4 min **Script:**

- **BSP cluster role:** This cluster answers the question "what does it mean for a barrier to exist and how do we compute one?" Stengle guarantees existence; Parrilo makes it computable; Lasserre ensures we can always escalate degree; Prajna & Jadbabaie wrote the exact three-obligation API we use.
- This slide establishes the theoretical lineage for the separation obligation $\forall s \in U : B(s) \leq -\epsilon$.
- **Stengle 1974 (Positivstellensatz):** if $\Phi(x) \wedge U(x)$ is infeasible over the reals, a polynomial identity proves it. This is the theoretical guarantee that a certificate must exist. We can cite this to a RISE audience as "the algebraic completeness guarantee under Archimedean conditions." Practical synthesis is the hard part, not existence.
- **Parrilo 2000 (SOS programming):** key insight is that a polynomial $p(x) \geq 0$ can be certified as a sum-of-squares decomposition $p = \sum q_i^2$, which reduces to a semidefinite program (SDP). This is the bridge from "certificate exists" to "we can compute one." Parrilo showed this via a reformulation over the Gram matrix.
- **Lasserre 2001 (moment/SOS hierarchy):** dual perspective using moment sequences. Iterative degree lifting of the SDP relaxation is guaranteed to converge to the global optimum (Lasserre hierarchy). Explains why we escalate degree when a low-degree certificate fails.
- **Prajna & Jadbabaie HSCC 2004:** this is the paper that writes down the exact three-obligation form we use. Before this, barrier/Lyapunov ideas were spread across control literature without a unified software-checkable interface. Stress: we use their triple almost verbatim over our symbolic machine state.
- **Prajna, Jadbabaie & Pappas TAC 2007:** stochastic extension. The consecution condition becomes an expectation inequality. Motivates our stochastic risk scoring path and hybrid certificate variants where execution has probabilistic branches.
- RISE point: the whole SOS chain (papers 1–5) answers the question "what does it mean for a certificate to exist and how do we compute one?" Everything downstream is about making that cheaper or more expressive.

12d. Separation tractability: relaxation ladder (papers 6–8)

Slide 12d 3 min **Script:**

- **BSP cluster role:** These papers make the separation check fast enough for CI without sacrificing soundness. B still satisfies all three obligations; only the proof of positivity is cheaper. Think of it as "the same fence, proved with cheaper paint." DSOS/SDSOS are sound subsets of SOS: they never produce a false certificate.
- Problem being solved: SOS/SDP is prohibitively expensive for high-degree or high-dimensional variable bases. The three papers here give us the cost ladder.
- **Ahmadi & Majumdar SIAM 2019 (DSOS/SDSOS):** DSOS replaces the SDP cone check with an LP check (diagonally dominant SOS); SDSOS uses SOCP. Both are sound subsets of SOS: they never certify something false. They are faster by orders of magnitude and are used as the *first tier* in the portfolio. Escalate to SDP only on DSOS failure. This is a key latency control decision for CI throughput.
- **Waki, Kim, Kojima & Muramatsu 2006 (correlative sparsity):** program states typically have sparse correlation graphs. If variable x_i and x_j never co-appear in the same constraint, their cross-terms can be factored out of the SDP. This reduces a single large SDP into many small coupled ones. Critical for any real Python function with 10+ live variables.
- **Ames et al. CBF survey 2019:** the continuous-system formulation where the consecution condition becomes $\dot{B}(x) \geq -\alpha(B(x))$ (a Lie derivative condition). This is informative for us because: (i) it motivates how a "slope" condition on B along transitions replaces a strict invariance requirement, and (ii) it shows where unknown-transition contracts enter: if we cannot compute $\dot{B}$ for an external call, we under-approximate the slope with a contract bound.
- RISE Q: "Why not just use SOS everywhere?" Answer: on synthetic tests, DSOS is 50–200x faster on the same problem. In CI, that is the difference between 10s and 30min per repo scan.

12e. Consecution obligation: property-directed reachability (papers 9–12)

Slide 12e 4 min **Script:**

- **BSP cluster role:** These four papers handle the consecution obligation when no polynomial template for B is available or affordable. Instead of finding B directly, they build an implicit barrier: IC3 and PDR represent $B \geq 0$ as a conjunction of learned clauses; CHC encodes consecution as a horn-clause fixpoint. The "fence" is the same concept, just represented symbolically.
- The consecution obligation asks: given $B(s) \geq 0$, does every successor also satisfy $B(s') \geq 0$? These four papers tackle this without needing a fixed polynomial template.
- **Sheeran, Singh & Stålmårck FMCAD 2000 (k-induction):** splits the consecution check into a bounded base (does any path of length $\leq k$ violate it?) and a step (does one more step violate it assuming the last k steps did not?). Used in the portfolio when the unsafe set is only reachable within bounded depth and blocked beyond. Maps to: after symbolic unrolling to depth k without finding a violation, promote candidate to "bounded-safe + induction step holding."
- **Bradley VMCAI 2011 (IC3):** instead of a fixed B, maintains a sequence of frame predicates $F_0 \supseteq F_1 \supseteq \ldots$ that over-approximate reachable sets at each time step. Blocking cubes: if a predecessor of a bad state is in F_i, push a lemma into F_{i-1} to exclude it. Convergence when $F_i = F_{i+1}$. This is purely about the consecution obligation: make the reachable-state sequence never touch U.
- **Een, Mishchenko & Brayton FMCAD 2011 (PDR):** industrial-scale IC3 implementation with generalization via ternary simulation and multi-property support. Shows that IC3/PDR scales to millions of state bits in hardware; our use is at much smaller scale but the same consecution-first logic applies. Key feature for us: PDR gives a UNSAT proof certificate (the inductive clause sequence) that we can serialize into the SARIF artifact.
- **Gurfinkel et al. CAV 2015 (SeaHorn):** models software verification as Constrained Horn Clause (CHC) satisfiability. A CHC system *is* the consecution obligation written interprocedurally: head predicates encode post-states, body clauses encode transitions + summaries. SeaHorn uses Eldarica and Z3's CHC engine. We map our interprocedural barrier check directly to this encoding. Tighter summaries = smaller CHC bodies = cheaper solving.
- RISE point: IC3/PDR and CHC are the methods that handle consecution when polynomial templates are wrong or unavailable.

12f. Template refinement: CEGAR and interpolants (papers 13–16)

Slide 12f 4 min **Script:**

- **BSP cluster role:** When a candidate B fails an obligation, these papers turn the failure into new template constraints. The obligation counterexample is not a dead end — it is a proof that the current monomial basis cannot represent the fence, and these methods extract the minimal new term needed to patch it.
- Problem: we have a spurious counterexample (a path that looks like it reaches U but does not in the real program). The certificate template failed. These papers give principled replies.
- **Clarke, Grumberg, Jha, Lu & Veith CAV 2000 (CEGAR):** the canonical loop. Abstract system $\rightarrow$ find counterexample $\rightarrow$ check if concrete $\rightarrow$ if spurious, refine abstraction. In our context: failed obligation with counterexample $\rightarrow$ add monomials to template basis $\rightarrow$ re-solve. CEGAR provides the termination argument: finite refinement steps until fixpoint or resource exhaustion.
- **Ball, Majumdar, Millstein & Rajamani PLDI 2001 (SLAM):** applied CEGAR to C using predicate abstraction for Windows driver verification. Key insight: predicates come from the counterexample trace (Boolean program predicates). Our analogue: path-condition inequalities from the spurious trace seed new template monomials.
- **McMillan CAV 2003 (Craig interpolants):** given a UNSAT proof of a spurious trace, extract a Craig interpolant between the prefix (init part) and suffix (unsafe part). The interpolant is a formula that (i) is implied by the prefix and (ii) contradicts the suffix. It is the strongest possible new predicate for refinement. Interpolants in our setup: if init-segment $\wedge \neg B \geq 0$ is UNSAT, extract interpolant to add to template constraints directly.
- **Henzinger, Jhala, Majumdar & Suresh POPL 2004 (lazy abstraction):** refine only the part of the abstract reachability tree that produced the counterexample. Everything else remains at the coarser abstraction. This maps to our targeted obligation retry: do not restart the full portfolio for a single spurious candidate; surgically enrich the failing template branch.
- RISE point: these four papers together answer "how do we systematically improve the template family without manual intervention?"

12g. Template synthesis and witness grounding (papers 17–20)

Slide 12g 4 min **Script:**

- **BSP cluster role:** SOS tells you what shape B must have; these papers propose specific coefficient values $c_{i,t}$ and then validate that the proposed B faithfully reflects the real program. Together they close the loop between "a fence shape exists" (Stengle) and "we have the right fence for this specific bug site" (concolic confirmation).
- Problem: who proposes the concrete $c_{i,t}$ values when SOS synthesis is stuck, and who confirms a candidate with a real execution trace?
- **Flanagan & Leino FME 2001 (Houdini):** start with an optimistically large set of candidate invariant clauses; run the verifier; drop any clause falsified by a counterexample; repeat until stable. In our portfolio: generate a large set of polynomial coefficient hypotheses, run obligation checks, and prune falsified ones. The fixpoint is the largest valid sub-certificate. Houdini-style pruning is cheap because counterexamples are fast to find.
- **Garg, Löding, Madhusudan & Neider CAV 2014 (ICE):** learn invariants from three classes of examples: *positive* (states known reachable and safe), *negative* (states known unsafe), and *implication* edges (state A implies state B is reachable). An ICE learner returns a classifier (invariant) consistent with all three. In our use: concrete execution traces provide positive/negative/implication examples; the learner proposes $c_{i,t}$ values for the barrier template. Especially powerful when symbolic synthesis is obstructed by nonlinearity.
- **Alur, Bodik, Juniwal et al. FMCAD 2013 (SyGuS):** syntax-guided synthesis fixes a grammar for the target formula and uses an SMT oracle to check candidates. In our portfolio: the barrier template family is the grammar; the SMT check of the three obligations is the oracle. SyGuS formalizes the solver-oracle loop we run and provides the completeness argument for the grammar-bounded search.
- **Godefroid, Klarlund & Sen PLDI 2005 (DART/concolic):** dynamic analysis + static analysis: execute program concretely while collecting path constraints symbolically, then negate a branch to produce new test inputs. In our portfolio: concolic runs (i) falsify candidate certificates by finding a violating execution, (ii) ground SAT models from symbolic stage into runnable witness traces, and (iii) confirm that a proposed initial set S_0 is reachable before committing the init obligation check.
- Key summary for RISE: papers 17–20 close the loop on the question SOS papers leave open. SOS tells you *what* shape a certificate must have; Houdini/ICE/SyGuS tell you *how to find the coefficients*; concolic tells you *whether the certificate is faithful to the real program*.

13. RISE detail: solver outcomes and pruning policy

Slide 13 2 min **Script:**

- UNSAT($\Phi \wedge U_t$): eliminate with proof artifact.
- SAT($\Phi \wedge U_t$): retain and possibly witness-seed.
- UNKNOWN/timeout: retain conservatively; escalate stage.
- This policy is central to soundness posture under practical solver limits.
- If asked: yes, this can preserve extra noise; that is the deliberate safety tradeoff.

14. Contracts for unknown/library calls

Slide 14 2 min **Script:**

- Pure determinism for unknown calls is unsound; pure nondeterminism is too noisy.
- Middle ground: conservative contracts with widening and fallback assumptions.
- Contract precision is a first-order determinant of FP/TP balance.
- Concolic checks can confirm some SAT survivors when contracts are loose.
- State clearly: this is an explicit assumption interface, not hidden magic.

15. CEGAR-style refinement

Slide 15 2 min **Script:**

- Coarse abstraction first for tractable throughput.
- Spurious survivors trigger targeted refinement of predicates/contracts/context.
- Re-check obligations after refinement, not ad-hoc confidence tweaking.
- This is how precision improves without globally exploding compute.

16. DSE/concolic as witness stage

Slide 16 2 min **Script:**

- Concolic runs ground symbolic possibilities into executable witnesses when feasible.
- Witness presence increases actionability confidence for true positives.
- Witness absence is not a safety proof; bounded search limits apply.
- Keep hierarchy explicit: proof artifacts dominate confidence artifacts.

17. Agentic-second triage

Slide 17 2 min **Script:**

- Triage inspects source ranges, guards, callers, imports, tests, and nearby patterns.
- It classifies only survivors that static stage could not prove/disprove within budget.
- Emphasize governance: triage does not invalidate proven UNSAT eliminations.
- Better formal evidence upstream usually improves triage quality downstream.

18. CI ratchet: not a firehose

Slide 18 2 min **Script:**

- Baseline historical findings once; block new unaccepted risk in CI.
- Auto-prune resolved findings for monotonic quality movement.
- This process model is often the difference between adoption and abandonment.
- Phrase: "ratchet turns infinite backlog into finite branch policy."

19. Soundness envelope and limits

Slide 19 2 min **Script:**

- Dynamic Python features and reflective behavior bound static completeness.
- Unknown external behavior remains contract-limited.
- Resource budgets impose incompleteness on hardest nonlinear/interprocedural cases.
- Strong claim to make: assumptions are explicit and failure modes are observable.

20. What RISE reviewers usually ask

Slide 20 2 min **Script:**

- **State encoding:** what is concrete vs symbolic?
- **Solver policy:** how are UNKNOWN and timeout treated?
- **Contracts:** where do unknown-call assumptions live and how are they audited?
- **Refinement:** what triggers CEGAR and when do you stop?
- **Artifact chain:** can every elimination be explained post-hoc?

21. Operational run modes

Slide 21 2 min **Script:**

- Quick demo mode: `a3 scan . --triage`
- Explicit CI mode: `a3 scan . --output-sarif results.sarif`, then `a3 triage --sarif results.sarif --agentic`
- Keep stage separation for debugging, traceability, and policy governance.
- For reproducibility in CI, pin model/provider and archive SARIF artifacts.

22. Close + Q&A

Slide 22 2 min **Script:**

- Re-state three claims: static-first trust, conservative escalation, ratchet adoption.
- Invite RISE discussion on soundness envelope, obligation encoding, and portfolio scheduling.
- Suggested starting point in practice: run static stage with SARIF evidence and baseline ratchet before enabling triage.
- Future work line: stronger contracts and richer nonlinear invariant synthesis under strict budgets.